CS:5980
Foundations of Embedded Systems

Liveness Requirements
Part II

Copyright 2014-26, Rajeev Alur and Cesare Tinelli.
Created by Cesare Tinelli at the University of Iowa from notes originally developed by Rajeev Alur at the University of Pennsylvania. These notes are copyrighted materials and may not be used in other course settings outside of the University of Iowa in their current form or modified form without the express written permission of one of the copyright holders. During this course, students are prohibited from selling notes to or being paid for taking notes by any person or commercial firm without the express written permission of one of the copyright holders.

LTL Recap

Syntax: Formulas built from

- Base formulas: Boolean-valued expressions over typed variables
- Logical connectives: $\wedge$, $\vee$, $\Rightarrow$, $\neg$, …
- Temporal Operators: Always, Eventually, Next, Until

Semantics: defined by rules for the satisfaction relation

- Formulas are evaluated w.r.t. a trace ρ (infinite sequence of valuations)
- A system satisfies spec φ iff every infinite execution satisfies φ

Derived operators:

Repeatedly _	(Always Eventually _)
Persistently _	(Eventually Always _)

Sample requirement: Every request is eventually granted

Always [req = 1 $\Rightarrow$ Eventually (grant = 1)]

Temporal Implications and Equivalences

Understanding subtle differences among different variants of LTL formulas can be tricky

Definition: Let φ_1, φ_2 be LTL formulas

1. φ_1 is *stronger* than φ_2 if every trace that satisfies φ_1 satisfies φ_2 too
   - i.e., every trace satisfies the implication $\varphi_1 \Rightarrow \varphi_2$
2. φ_1 is equivalent to φ_2 if φ_1 and φ_2 are satisfied by exactly the same traces
   - i.e., each formula is stronger than the other
   - i.e., every trace satisfies the double implication $\varphi_1 \Leftrightarrow \varphi_2$
   - i.e., the two formulas express exactly the same requirement

Knowing some standard equivalences is useful for simplifying formulas

Temporal Implications and Equivalences

- Always φ is stronger than φ
- Repeatedly φ is equivalent to $\neg$Persistently $\neg\varphi$
- Persistently φ is stronger than Repeatedly φ
- Always φ is equivalent to $\varphi \wedge$ Next Always φ
- Eventually φ is equivalent to $\varphi \vee$ Next Eventually φ
- Always φ is equivalent to $\neg$Eventually $\neg\varphi$

Exercise: What is the mutual relationship between these formulas?

1. Always Eventually φ
2. Next Always Eventually φ
3. Eventually Always Eventually φ

Logical Connectives and Temporal Operators

Are these two formulas equivalent?

Eventually $(\varphi_1 \vee \varphi_2)$ Eventually $\varphi_1 \vee$ Eventually φ_2 Yes, they are

Proof:

⇒) Suppose a trace ρ satisfies Eventually $(\varphi_1 \vee \varphi_2)$

- There is a position j such that $(\rho, j) \vDash \varphi_1 \vee \varphi_2$
- Either $(\rho, j) \vDash \varphi_1$ or $(\rho, j) \vDash \varphi_2$
- Suppose $(\rho, j) \vDash \varphi_1$ (the other case is similar)
- Then ρ satisfies Eventually φ_1
- Hence, it also satisfies Eventually $\varphi_1 \vee$ Eventually φ_2

⇐) Suppose a trace ρ satisfies Eventually $\varphi_1 \vee$ Eventually φ_2

- Suppose ρ satisfies Eventually φ_1 (the other case is similar)
- There is a position j such that $(\rho, j) \vDash \varphi_1$
- Then, $(\rho, j) \vDash \varphi_1 \vee \varphi_2$
- It follows that ρ satisfies Eventually $(\varphi_1 \vee \varphi_2)$

Logical Connectives and Temporal Operators

Are these two formulas equivalent?

Eventually $(\varphi_1 \wedge \varphi_2)$ Eventually $\varphi \wedge$ Eventually φ_2

The first is stronger than the second but not vice versa

Proof:

$\Rightarrow$) Suppose a trace ρ satisfies Eventually $(\varphi_1 \wedge \varphi_2)$

- There exists a position j such that $(\rho, j) \vDash \varphi_1 \wedge \varphi_2$
- It follows that both $(\rho, j) \vDash \varphi_1$ and $(\rho, j) \vDash \varphi_2$
- Since $(\rho, j) \vDash \varphi_1$ then ρ satisfies Eventually φ_1
- Similarly, it also satisfies Eventually φ_2
- It follows that ρ satisfies Eventually $\varphi_1 \wedge$ Eventually φ_2

$\Leftarrow$) To disprove this, consider trace 0,1,0,1,0,1,… over a Boolean variable x

- Trace satisfies Eventually $(x = 0) \wedge$ Eventually $(x = 1)$
- But does not satisfy Eventually $(x = 0 \wedge x = 1)$

Logical Connectives and Temporal Operators

Distributivity rules for logical connectives and temporal operators

Exercise: Are these pairs of formulas equivalent?

1. Always $(\varphi_1 \wedge \varphi_2)$ Always $\varphi_1 \wedge$ Always φ_2
2. Always $(\varphi_1 \vee \varphi_2)$ Always $\varphi_1 \vee$ Always φ_2
3. Repeatedly $(\varphi_1 \wedge \varphi_2)$ Repeatedly $\varphi_1 \wedge$ Repeatedly φ_2
4. Repeatedly $(\varphi_1 \vee \varphi_2)$ Repeatedly $\varphi_1 \vee$ Repeatedly φ_2

Back to Fairness

Weak fairness: An infinite execution is *fair* to a task A if, repeatedly, either A is executed or is disabled

If task is enabled, then it is eventually executed or disabled

Strong fairness: An infinite execution is *fair* to a task A, if task A is either executed repeatedly or disabled continuously from a certain step onwards

If task is repeatedly enabled, then it is repeatedly executed

Back to Fairness

Process P

nat x := 0; bool y := 0
A: x := x + 1
B: even(x) –> y := 1 - y

What fairness assumptions are needed so that P satisfies the spec

1. Eventually ($x \geq 10$) :
   weak fairness for A
2. Eventually ($y = 1$) :
   strong fairness for B

Back to Fairness

Process P

nat x := 0; bool y := 0
A: x := x + 1
B: even(x) –> y := 1 - y

Fairness can be encoded directly in LTL!

Instead of checking if the system satisfies an LTL formula φ, check if it satisfies the formula

$$\text{FairnessAssumption} \Rightarrow \varphi$$

FairnessAssumption is an LTL formula encoding what it means for an execution to be weakly/strongly fair with respect to a task

Encoding Weak Fairness in LTL

Process P

nat x := 0; bool y := 0; { A, B } executed
A: x := x + 1; executed := A
B: even(x) –> y := 1 - y; executed := B

1. We add a variable called executed whose values are task names
2. Whenever a task executes, executed is assigned the name of the task

Weak fairness for a task T: An execution is weakly fair to task T if it satisfies the formula

WF_T: Persistently (T is enabled) ⇒ Repeatedly (T is executed)

Examples:

WF_A: Repeatedly (executed = A)

WF_B: Persistently (even(x)) ⇒ Repeatedly (executed = B)

Checking Requirements under Weak Fairness

Process P

nat x := 0; bool y := 0; {A, B} executed
A: x := x + 1; executed := A
B: even(x) –> y := 1 - y; executed := B

Which of the following are satisfied by P ?

1. Eventually ($x \geq 10$)
2. $WF_A \Rightarrow$ Eventually ($x \geq 10$)
3. $WF_B \Rightarrow$ Eventually ($y = 1$)
4. $(WF_A \wedge WF_B) \Rightarrow$ Eventually ($y = 1$)

What have we achieved?

- Checking if an LTL spec is satisfied under fairness assumptions is reduced to checking a modified LTL spec
- Then the verifier does not have to handle fairness explicitly

Encoding Strong Fairness

Process P

nat x := 0; bool y := 0; {A, B} executed
A: x := x + 1;   executed := A
B: even(x) –> y := 1 - y; executed := B

Strong fairness for a task T: An infinite execution is strongly fair to task T if it satisfies the formula

SF_T: Repeatedly (T is enabled) ⇒ Repeatedly (T is executed)

Example:

SF_B: Repeatedly (even(x)) ⇒ Repeatedly(executed = B)

Note: if a spec is satisfied assuming weak fairness, then it also satisfied assuming strong fairness

Encoding Strong Fairness

Process P

nat x := 0; bool y := 0; {A,B} executed
A: x := x + 1; executed := A
B: even(x) –> y := 1 - y; executed := B

Strong fairness for a task T: An infinite execution is strongly fair to task T if it satisfies the formula

SF_T: Repeatedly (T is enabled) ⇒ Repeatedly (T is executed)

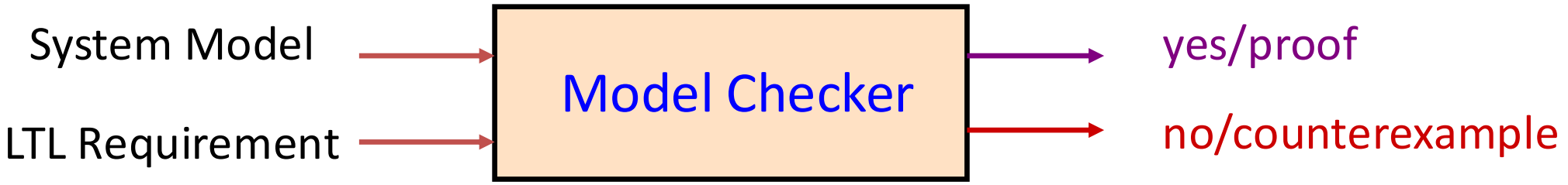
Example:

SF_B: Repeatedly (even(x)) ⇒ Repeatedly(executed = B)

Exercise: Which of the following specs are satisfied by P?

1. SF_B ⇒ Eventually (y = 1)
2. SF_B ⇒ Repeatedly (y = 1)
3. SF_B ⇒ Persistently (y = 1)

Model Checking



- Performed using enumerative or symbolic search through the state-space of the program
- Success story for transitioning academic research to industrial practice
- Used to debug multi-core protocols, pipelined processors, device driver code, distributed algorithms in Intel, Microsoft, IBM …

Büchi Automata

A safety monitor M classifies finite executions into good and bad

Verification of safety requirements for a component C reduces to analyzing reachable states of the composition of C and M

An error execution is an execution that leads the monitor into an error state

How can a monitor (aka, an automaton) classify infinite executions into good and bad?

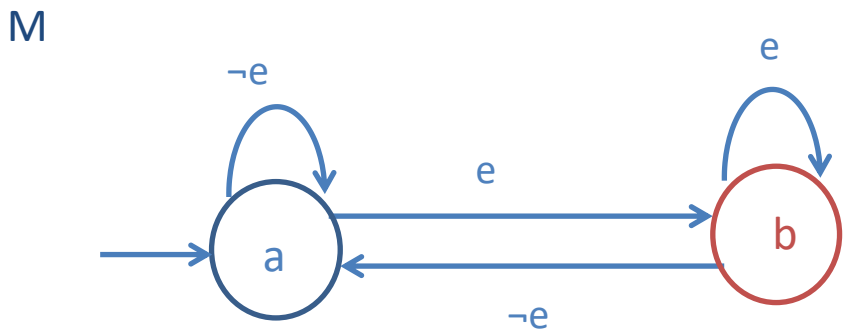
Büchi Automata

Theoretical model of Büchi automata proposed by Richard Büchi (1960)

Model checking application (1990s) using Büchi automata:

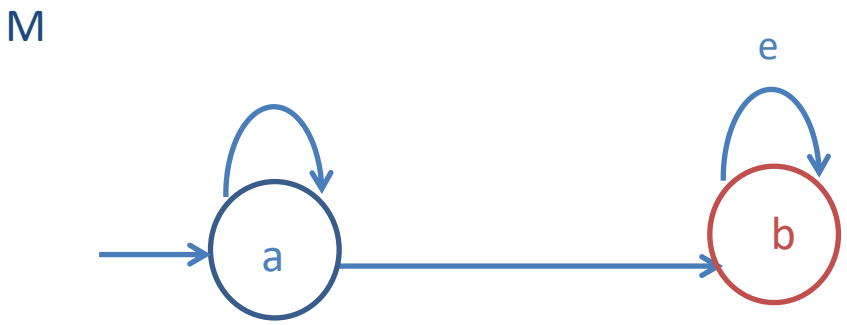
- Automatically translate LTL formula φ to a Büchi monitor M
- Consider the composition of system C and monitor M
- Reachable cycles in this composite correspond to counter-examples; if no such cycle is found, system satisfies spec
- Implemented in many model checkers (notably, SPIN)

Büchi Automaton: Example 1



- **Inputs:** Boolean variable e
- Of two states a and b, a is initial and b is accepting
- Given a trace ρ over e (i.e., infinite sequence of 0/1 values to e), there is a corresponding execution of M
- The trace ρ is accepted if accepting state appears repeatedly
- Language of M = { traces in which e is satisfied repeatedly }
- M accepts ρ iff ρ ⊨ Repeatedly e

Büchi Automaton: Example 2



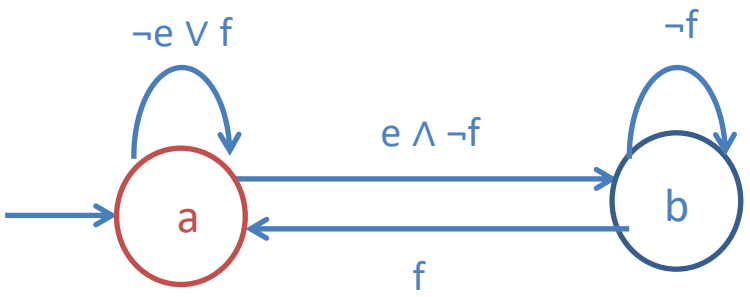
- Automaton is nondeterministic
- On a given input trace, many possible executions
- An execution is accepting if it visits accepting state repeatedly
- M accepts an input trace ρ if there exists some accepting execution on that input
- M accepts ρ iff $\rho \models$ Persistently e

Büchi Automaton: Example 3

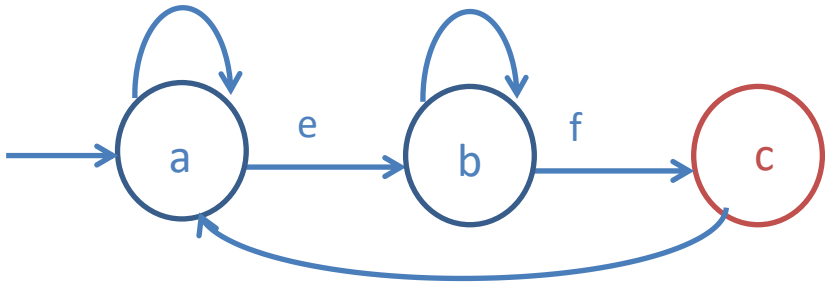
- Design a Büchi automaton M such that

  M accepts trace ρ iff $\rho \models$ Always (e $\Rightarrow$ Eventually f)

- Inputs: Boolean values for e and f
- In an accepting execution, every e must be followed by f



Büchi Automaton: Example 4



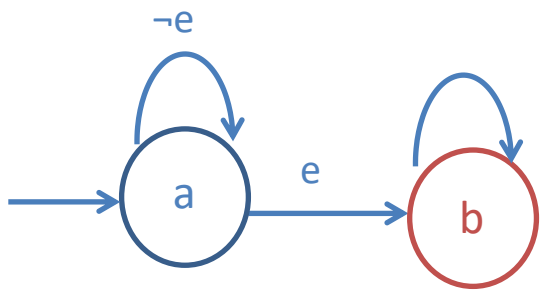
Which traces does this accept? Express it in LTL

M accepts ρ iff $\rho \models$ Repeatedly e $\wedge$ Repeatedly f

Büchi Automaton M Definition

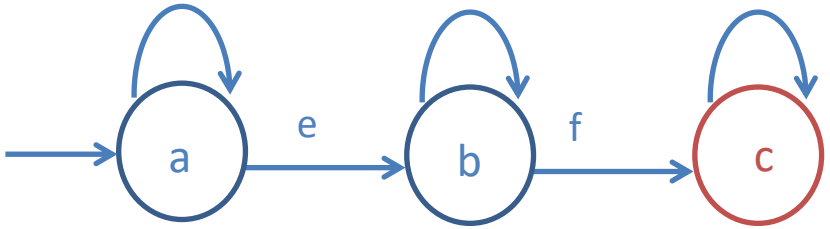
- Set of Boolean V of input variables
- Finite set Q of states
- Set Init of initial states
- Set F of accepting states
- Set of edges/transitions of the form $q \text{ –G–> } q'$
  where G is a Boolean-valued condition over V
- Given an input trace $\rho = v_1, v_2, v_3, \ldots$ over V, an accepting execution of M over ρ is an infinite sequence of states $q_0, q_1, q_2, \ldots$ where
  - q_0 is initial
  - For each i, there is an edge $q_i \text{ –G–> } q_{i+1}$ such that input v_i satisfies G
  - There are infinitely many positions i such that state q_i is in F
- M accepts input trace ρ if there is an accepting execution of M over ρ

Büchi Automata: More Examples



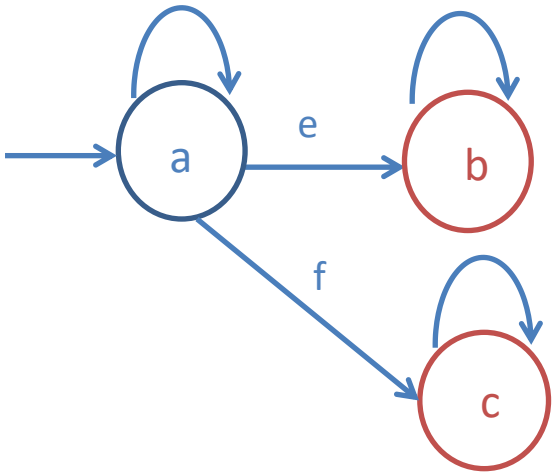
Eventually e

Büchi Automata Examples

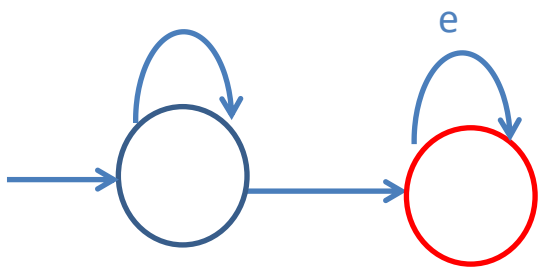


Eventually [e ∧ Next Eventually f]

Eventually e ∨ Eventually f



Nondeterministic Büchi Automaton

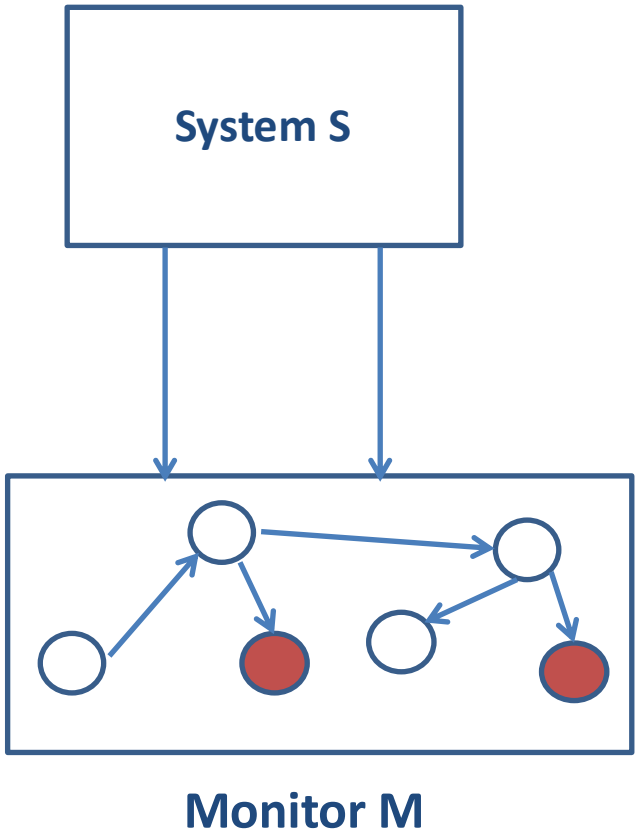


Persistently e

Can we construct an equivalent deterministic Büchi automaton?

No! Non-determinism is sometimes necessary!

Safety Monitors



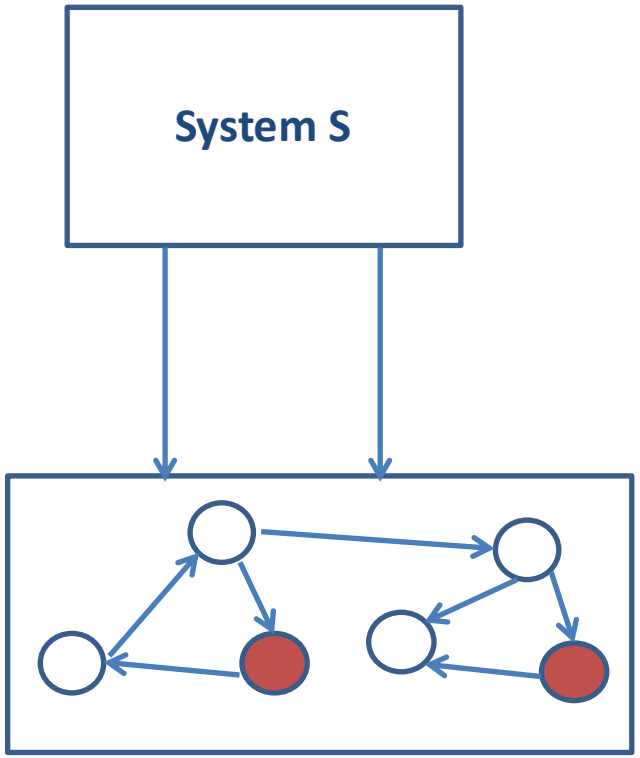
Is there an execution of **S** that makes **M** enters an error state?

M is designed so that such an execution indicates a bug!

Verification reduces to reachability

Check if an error state is reachable in composition of **S** and **M**

Büchi Monitors



Büchi Monitor M

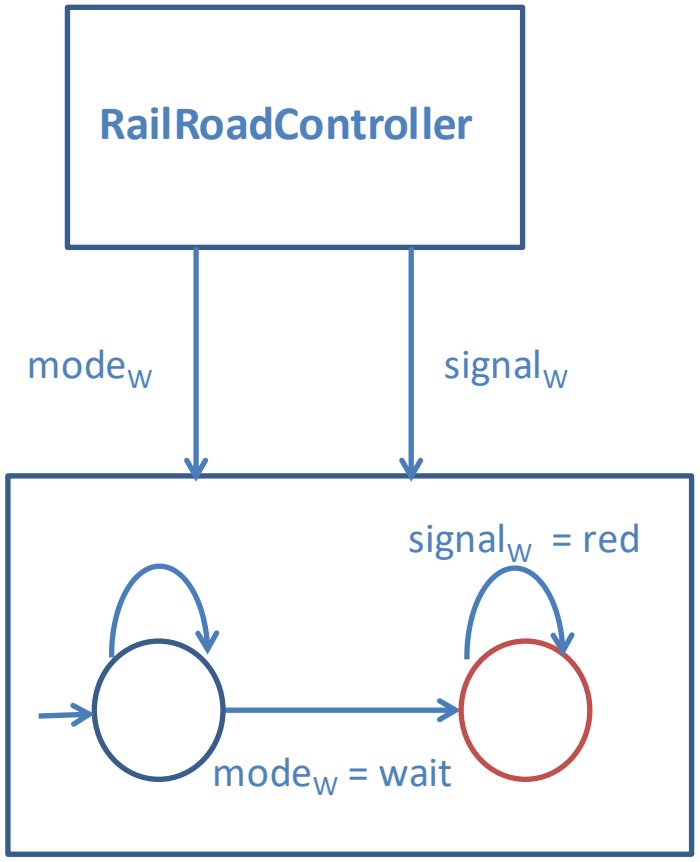
Is there an infinite execution of **S** which is accepted by **M**? (i.e., an execution in which some error state of **M** appears repeatedly?)

M is designed so that such an execution indicates a bug!

Verification reduces to search for cycles

Check if there is a reachable cycle containing an error state in the composition of **S** and **M**

Example Büchi Monitor



Büchi Monitor M

Correctness requirement:

Always [(West train is waiting) ⇒
Eventually (West signal is green)]

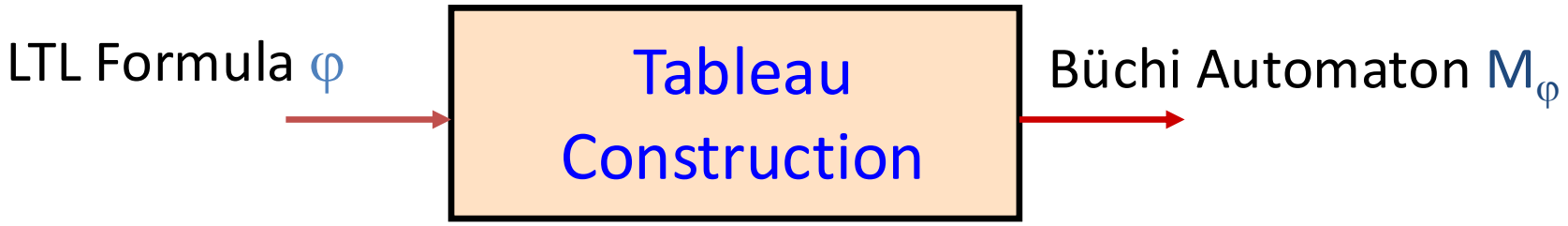
Requirement violation:

Infinite execution where, at some step, west train is waiting and in all subsequent times west signal is red

Verification:

Search for reachable cycle containing red monitor state in the combined system

From LTL to Büchi Automata



Automaton M_φ accepts exactly those traces that satisfy formula φ

To check if a system C satisfies the LTL requirement φ

- construct the Büchi automaton M_φ corresponding to φ
- search for cycles in composition of C and M_φ

Tableau Construction Example

Consider Always e ∧ Eventually f: A e ∧ E f



A state is a collection of formulas that must be satisfied

Initial state contains given formula

Formulas in a state must be consistent with rules of logical connectives: for example, if a state has $\varphi_1 \wedge \varphi_2$, then it must have both φ_1 and φ_2

Omega-Regular Languages

- The *language* of a Büchi automaton is the set of traces it accepts
- Such languages are called *ω-regular*

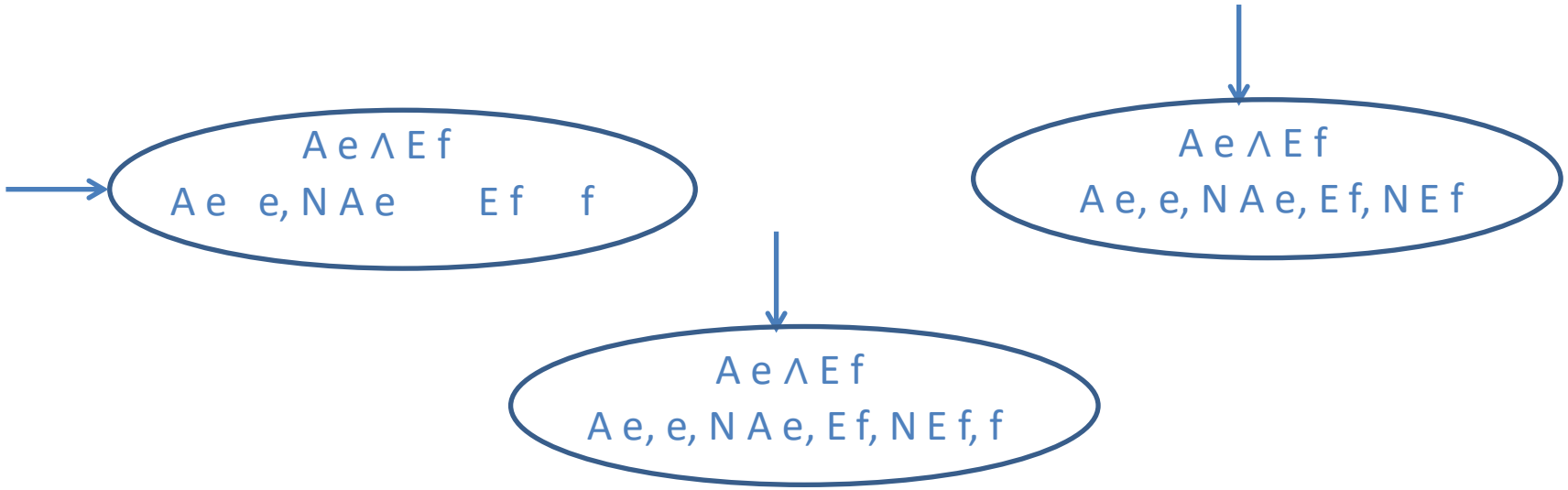
- There is a well-developed theory of ω-regular languages

- It is analogous the classical theory of regular languages
  (i.e., languages of finite strings of input characters accepted by finite automata)

- **Relevance to us:** Given an LTL formula φ, there is an algorithm to construct a Büchi automaton M_φ that accepts exactly the traces that satisfy φ

Tableau Construction Example

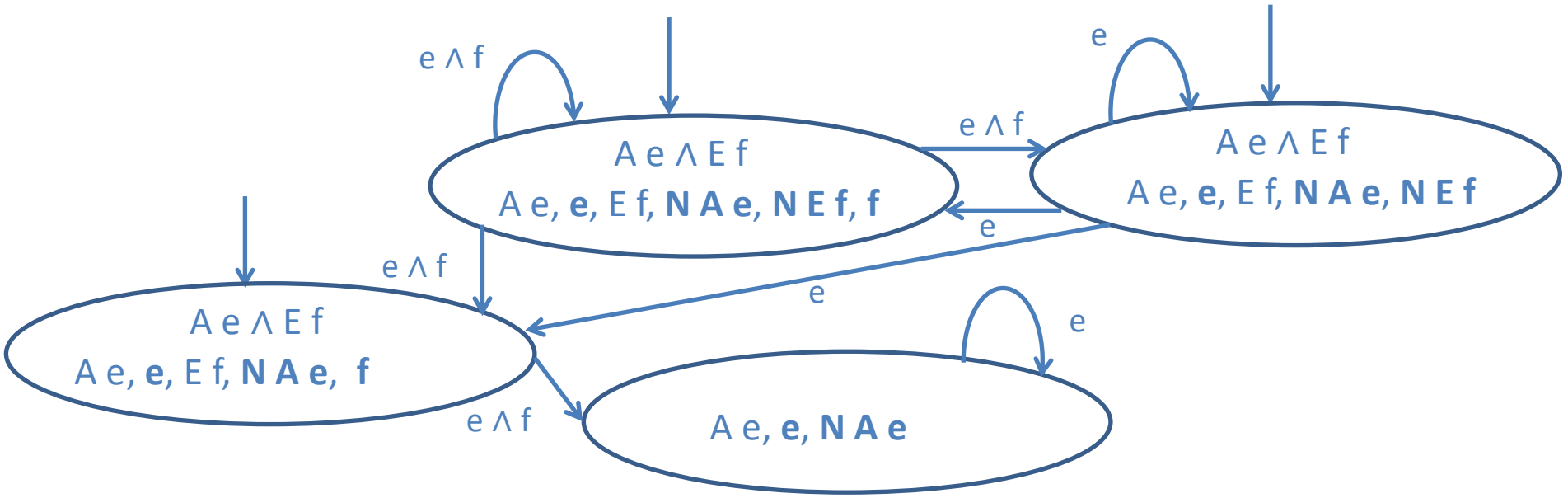
Consider Always e ∧ Eventually f: A e ∧ E f



- If a state has Always φ, it must have both φ and Next Always φ
- If a state has Eventually φ, it must have either φ or Next Eventually φ or both
  This leads to 3 cases

Tableau Construction Example

Consider Always e ∧ Eventually f: A e ∧ E f

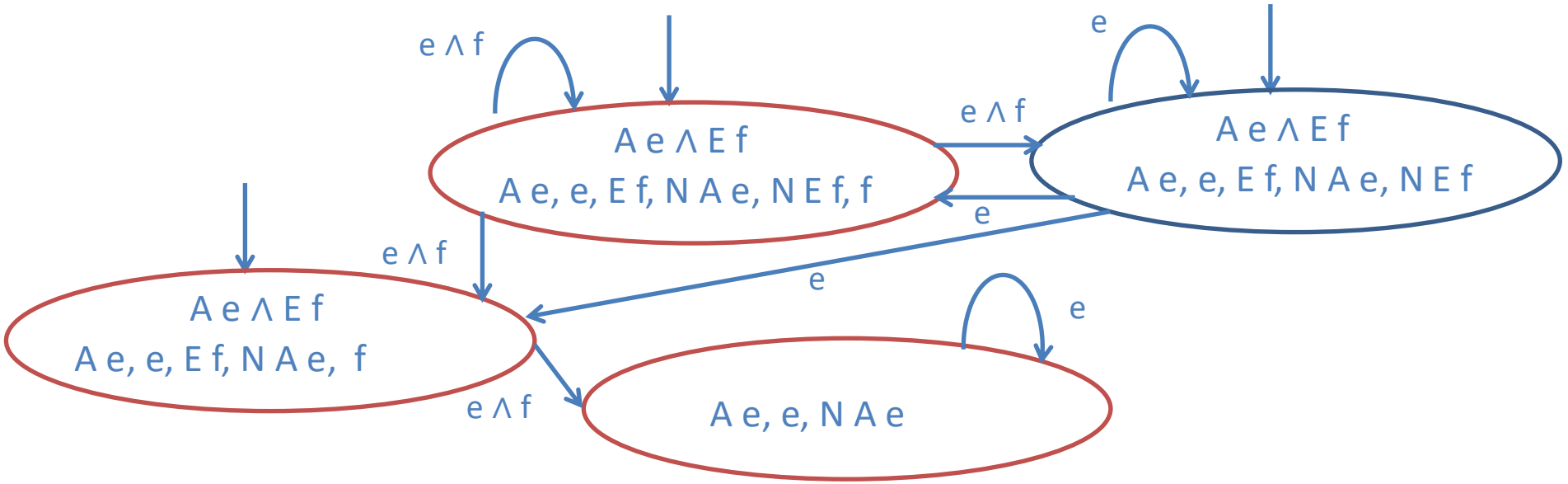


Transition Rules:

1. If a state contains Next φ then add transition to each state containing φ
2. If a state contains base formula φ, then φ must hold on outgoing transitions

Tableau Construction Example

Consider Always e ∧ Eventually f: A e ∧ E f

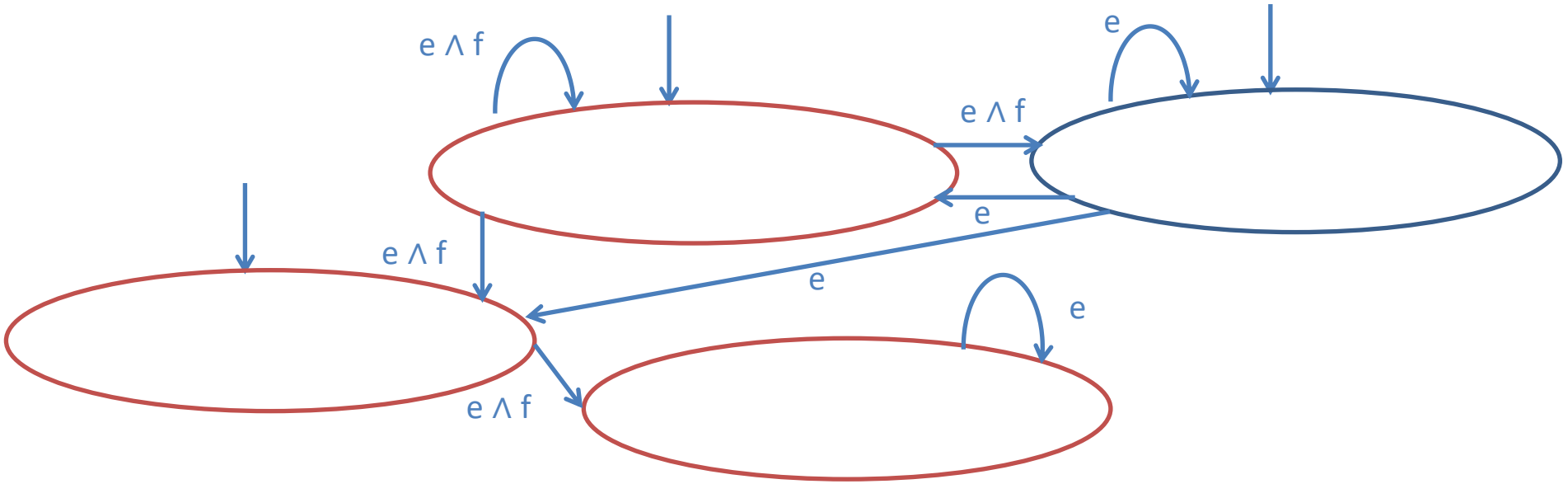


Acceptance condition: Satisfaction of eventuality should not be postponed forever

Accepting states: States that either contain f or do not contain E f

Tableau Construction Example

Consider Always e ∧ Eventually f: A e ∧ E f

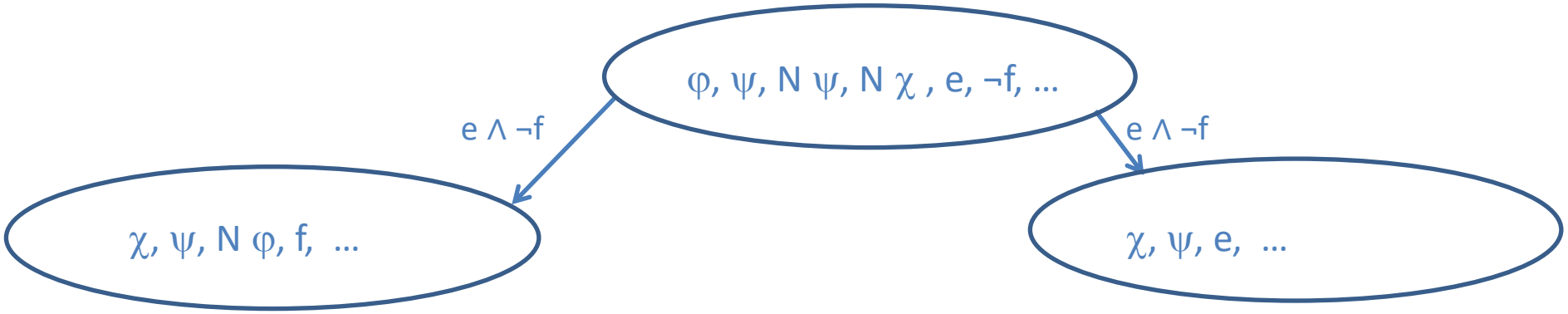


Indeed, this is a correct Büchi automaton for the given formula!

Tableau Construction Overview

Automaton/tableau state: Collection of relevant LTL formulas

Intended meaning: All the formulas in a state must hold on every infinite path starting at a state



Local consistency rules ensure that for every non-atomic formula φ, the state contains additional formulas ensuring that φ holds

Transition rules ensure that

1. every atomic formula holds at current time, and
2. all Next formulas are propagated to next state

Formal Construction

Given an LTL-formula φ, define set Sub(φ), the *closure* of φ

Sub(φ) consists of formulas that are relevant to evaluation of φ:

- It contains all the subformulas of φ
- If it contains Always ψ, it also contains Next Always ψ
- If it contains Eventually ψ, it also contains Next Eventually ψ
- If it contains φ_1 U φ_2, it also contains Next (φ_1 U φ_2)

Example:

Sub(Always Eventually e $\wedge$ Next f) =

{ Always Eventually e $\wedge$ Next f,
Always Eventually e, Eventually e, e
Next f, f,
Next Eventually e, Next Always Eventually e }

Note: Size of Sub(φ) is linear in the size of φ

Tableau States

A state of the desired automaton is a subset of Sub(φ) that satisfies some consistency rules:

- Does not contain both a formula φ_1 and its negation $\neg\varphi_1$
- Contains $\varphi_1 \wedge \varphi_2$ exactly when it contains both φ_1 and φ_2
- Contains $\varphi_1 \vee \varphi_2$ exactly when it contains at least one of φ_1 and φ_2
- Contains $\varphi_1 \Rightarrow \varphi_2$ exactly when it contains at least one of $\neg\varphi_1$ and φ_2
- If it contains Always φ_1 then it contains both φ_1 and Next Always φ_1
- If it contains Eventually φ_1 then it contains at least one of φ_1 and Next Eventually φ_1
- If it contains φ_1 U φ_2, it contains φ_1 or both φ_2 and Next (φ_2 U φ_2)

Note: Number of possible states is exponential in the size of φ

Example Construction

Formula φ = Eventually e $\wedge$ Next $\neg$e

Sub(φ) = { E e $\wedge$ N $\neg$e, E e, e, N $\neg$e, $\neg$e, N E e }

Tableau states:

q_0 = { e, N $\neg$e, N E e, E e, E e $\wedge$ N $\neg$e }

q_1 = { e, N E e, E e }

q_2 = { e, N $\neg$e, E e, E e $\wedge$ N $\neg$e }

q_3 = { e, E e }

q_4 = { $\neg$e, N $\neg$e, N E e, E e, E e $\wedge$ N $\neg$e }

q_5 = { $\neg$e, N E e, E e }

q_6 = { $\neg$e, N $\neg$e }

q_7 = { $\neg$e }

Tableau Construction Continued

Input variables V**:** base formulas appearing in φ

States: Consistent subsets of Sub(φ)

Initial states: States that contain the formula φ

Transitions: q –G–> q’ is a transition provided

- Next ψ is in q exactly when ψ is in q’
- If a base formula e is in q, then e is a conjunct in G, else ¬e is a conjunct in G

Example Construction Continued

Formula φ = Eventually e $\wedge$ Next $\neg$e

Tableau states:

q_0 = { e, N $\neg$e, N E e, E e, E e $\wedge$ N $\neg$e }

q_1 = { e, N E e, E e }

q_2 = { e, N $\neg$e, E e, E e $\wedge$ N $\neg$e }

q_3 = { e, E e }

q_4 = { $\neg$e, N $\neg$e, N E e, E e, E e $\wedge$ N $\neg$e }

q_5 = { $\neg$e, N E e, E e }

q_6 = { $\neg$e, N $\neg$e }

q_7 = { $\neg$e }

Transitions from q_0:

q_0 –e–> q_4

q_0 –e–> q_5

Transitions from q_1:

q_1 –e–> q_0

q_1 –e–> q_1

q_1 –e–> q_2

q_1 –e–> q_3

Transitions from q_6:

q_6 –($\neg$e)–> q_6

q_6 –($\neg$e)–> q_7

Tableau Construction: Acceptance

For a subformula Eventually ψ, need to ensure that satisfaction of ψ is not postponed forever.

Whenever Eventually ψ appears is in a state either ψ or Next Eventually ψ, or both, are included

Define F to be the set of tableau states that either include ψ or exclude Eventually ψ

Accepting condition: Repeatedly F

Similarly, for a subformula Always ψ,

1. Define F' to be the set of states that either include Always ψ or exclude ψ
2. A state in F' is required to appear repeatedly on an accepting run

Example Construction Continued

Formula φ = Eventually e $\wedge$ Next $\neg$e

Tableau states:

q_0 = { e, N $\neg$e, N E e, E e, E e $\wedge$ N $\neg$e }

q_1 = { e, N E e, E e }

q_2 = { e, N $\neg$e, E e, E e $\wedge$ N $\neg$e }

q_3 = { e, E e }

q_4 = { $\neg$e, N $\neg$e, N E e, E e, E e $\wedge$ N $\neg$e }

q_5 = { $\neg$e, N E e, E e }

q_6 = { $\neg$e, N $\neg$e }

q_7 = { $\neg$e }

Accepting states:

{ q_0, q_1, q_2, q_3, q_6, q_7 }

Initial states: { q_0, q_2, q_4 }

Transitions from q_0:

q_0 –e–> q_4

q_0 –e–> q_5

Transitions from q_1:

q_1 –e–> q_0

q_1 –e–> q_1

q_1 –e–> q_2

q_1 –e–> q_3

Transitions from q_6:

q_6 –($\neg$e)–> q_6

q_6 –($\neg$e)–> q_7

Handling Acceptance

In general, if there are multiple temporal formulas, the acceptance condition should ensure that each is satisfied

Generalized Büchi Automaton: Modest syntactic generalization

Automaton M has k accepting sets F_1, F_2, … F_k

An execution is accepting if for each j, some state in F_j appears repeatedly

$$\text{Repeatedly } F_1 \wedge \text{Repeatedly } F_2 \wedge \ldots \wedge \text{Repeatedly } F_k$$

- It is possible to *compile* a generalized Büchi automaton to a standard Büchi automaton
- It is also possible to adapt cycle-detection algorithms to handle multiple accepting sets

Tableau Construction: Summary

Correctness: A trace over V satisfies a given LTL formula φ iff it is accepted by the Generalized Büchi Automaton M_φ

Complexity: Size of M_φ is 2^n, where n is the size of φ (such a blow-up is unavoidable)

Practical implementations with a number of optimizations exist

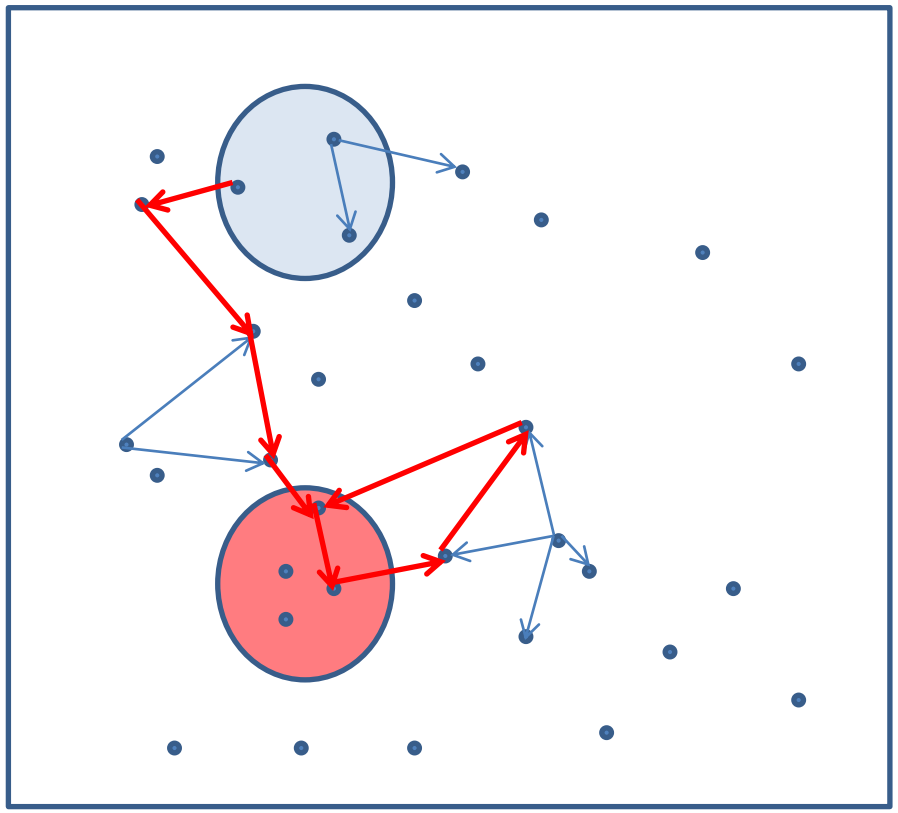
Reachability Problem for Transition Systems



- Is there a (finite) execution from an initial state to a state satisfying φ
- Checking whether φ is an invariant of T reduces to checking if ¬φ is reachable
- Verification techniques
  1. Proof-based (inductive invariants)
  2. Enumerative on-the-fly search (see notes)
  3. Symbolic search based on iterative image computation

Repeatable Property for Transition Systems

Transition System = (States, Initial states, Transitions)



Property φ : Subset of states

Property φ is *repeatable* if there exists an infinite execution that satisfies Repeatedly φ

Is there a state s such that
1. s is reachable
2. s satisfies φ
3. there is a cycle containing s

Repeatability Problem for Transition Systems



Is there an infinite execution along which states satisfying φ appear repeatedly?

To check whether a system C satisfies an LTL formula φ, check if property Mode is accepting is repeatable in composition of C and Büchi monitor $M_{\neg\varphi}$

Verification techniques (not covered, see Chap 5)

1. Proof-based: Ranking functions
2. Enumerative: Nested Depth-first Search
3. Symbolic search

Credits

Notes based on Chapter 5 of

Principles of Cyber-Physical Systems
by Rajeev Alur
MIT Press, 2015